

Lógica de Programação

Prof.^a Kecia Aline Marques Ferreira

DECOM | Departamento de
Computação



Lógica de Programação

Funções

Funções

- Vetores como parâmetros
- Matrizes como parâmetros
- Protótipos de funções
- Escopo
- Modularidade

Vetores como Parâmetros

Vetores como Parâmetros

É possível passar um vetor como parâmetro para um função

Um forma de se fazer isso é:

- Na declaração da função

tipo_de_retorno nome_da_função (tipo nome_vetor[tamanho])

ou

tipo_de_retorno nome_da_função (tipo nome_vetor[])

- Na chamada da função

nome_da_função (nome_vetor)

Vetores como Parâmetros

Quando um vetor é passado como argumento, ele sofre as alterações que forem realizadas no parâmetro dentro da função

Vetores como Parâmetros

```
#include <stdio.h>
#define TAM 10

void imprimeVetor(int vet[]){
    for (int i=0; i<TAM; i++)
        printf("- %d", vet[i]);
}

void leVetor(int vet[]){
    for (int i=0; i<TAM; i++)
        scanf("%d", &vet[i]);
}

void main(){
    int vetor[10];
    leVetor(vetor);
    imprimeVetor(vetor);
}
```

Vetores como Parâmetros

Pode-se incluir o tamanho do vetor na definição da função

Exemplo:

```
void imprimeVetor(int vet[TAM]) {  
    for (int i=0; i<TAM; i++)  
        printf("- %d", vet[i]);  
}
```


Vetores como Parâmetros

Existe uma outra forma de se passar vetor como parâmetros utilizando-se o conceito de **ponteiro**

Posteriormente, veremos essa forma.

Matrizes como Parâmetros

Matrizes como Parâmetros

É possível passar uma matriz como parâmetro para um função

Um forma de se fazer isso é:

- Na declaração da função

**tipo_de_retorno nome_da_função (tipo
nome_matriz[linhas][colunas])**

- Na chamada da função

nome_da_função (nome_matriz)

Matrizes como Parâmetros

Quando uma matriz é passada como argumento, ela sofre as alterações que forem realizadas no parâmetro dentro da função

Matrizes como Parâmetros

```
#include <stdio.h>
#define LIN 3
#define COL 2

void imprimeMatriz(int mat[LIN][COL]){
    for (int i=0; i<LIN; i++){

        for (int j=0; j<COL; j++)
            printf("- %d", mat[i][j]);

        printf("\n");
    }
}

void leMatriz(int mat[LIN][COL]){
    for (int i=0; i<LIN; i++)
        for (int j=0; j<COL; j++)
            scanf("%d", &mat[i][j]);
}
```

Matrizes como Parâmetros

```
void main() {  
    int matriz[LIN][COL];  
  
    leMatriz(matriz);  
    imprimeMatriz(matriz);  
}
```

Matrizes como Parâmetros

Pode-se passar uma matriz como parâmetro utilizando-se o conceito de **ponteiro**

Posteriormente, veremos essa forma.

Protótipos de Funções

Protótipos de Funções

Quando uma função é chamada, os argumentos devem ter tipos compatíveis com os parâmetros esperados pela função

O compilador é quem faz essa verificação de compatibilidade de tipos

Para que a verificação seja feita, é necessário que a função seja definida no programa antes do seu uso

Protótipos de Funções

```
void funcao1(char x) {  
    //corpo da função ...  
}
```

```
int funcao2(float b) {  
    //corpo da função ...  
    funcao1('A');  
}
```

Protótipos de Funções

Pode-se, porém, declarar todas as funções no início do programa e, posteriormente no código fonte, defini-las, ou seja, implementar os seus respectivos corpos

Essa declaração é denominada protótipo de funções

Protótipos de Funções

```
#include <stdio.h>
#define LIN 3
#define COL 2

void imprimeMatriz(int mat[LIN][COL]);
void leMatriz(int mat[LIN][COL]);

void main() {
    int matriz[LIN][COL];

    leMatriz(matriz);
    imprimeMatriz(matriz);
}
```

Protótipos de Funções

```
void imprimeMatriz(int mat[LIN][COL]) {
    for (int i=0; i<LIN; i++){

        for (int j=0; j<COL; j++)
            printf("- %d", mat[i][j]);

        printf("\n");
    }
}

void leMatriz(int mat[LIN][COL]) {
    for (int i=0; i<LIN; i++)
        for (int j=0; j<COL; j++)
            scanf("%d", &mat[i][j]);
}
```

Escopo

Escopo

Um região de um código em C corresponde a um **bloco**

Em C, um bloco de comando é **delimitado por chaves** ou corresponde a um comando de fluxo de controle

Escopo é a região do código em que um nome (uma variável, por exemplo) é conhecida

Escopo

Quando uma variável é declarada dentro de um bloco, ela tem escopo local naquele bloco

ou seja

A variável só é conhecida dentro daquele bloco

Escopo

Exemplo:

```
for (int i=0; i<10; i++)  
    printf("- %d", i);
```

Neste trecho, a variável *i* é local ao bloco *for*

Ou seja, ao sair do bloco *for*, a variável *i* deixa de existir

Escopo

Exemplo:

```
int i;  
for (i=0; i<10; i++)  
    printf("- %d", i);
```

Nesse outro exemplo, a variável *i* foi declarada fora do bloco *for*

Ou seja, ao sair do bloco *for*, a variável *i* continuar a existir

Escopo

Uma função também é um bloco, pois é delimitada por chaves

Os nomes declarados na função (nomes dos parâmetro e variáveis) têm **escopo local à função**

Ou seja, eles são conhecidos dentro de qualquer parte da função, mas não são conhecidos fora da função

Diz-se que é uma **variável local**

Escopo

```
#include <stdio.h>
```

```
int soma(int a, int b) {  
    int result;  
    result = a + b;  
    return result;  
}
```

a e **b** são os parâmetros, portanto, são locais da função *soma*
result é uma variável local da função *soma*

```
void main() {
```

```
    int a=10, b=5;  
    printf("%d", soma(a,b) );
```

Essas variáveis **a** e **b** são locais da função *main*

```
}
```

Escopo

Atenção!

Será mostrado agora o conceito de **variável global**

Porém, **variáveis globais devem ser evitadas**, pois são fonte de erros em programas e dificulta a manutenção de programas

Escopo

Uma variável global tem escopo global no programa, ou seja, é conhecida no programa inteiro

Uma variável global é definida fora das funções, inclusive da *main*

Isso significa que ela pode ser lida e alterada por qualquer parte do programa

Esse é o perigo! Quando há um erro no programa em decorrência de um valor errado atribuído à variável global, é difícil identificar a parte do programa que a alterou incorretamente.

Portanto, a regra de ouro é: **evite usar variável global!**

Escopo

```
#include <stdio.h>
```

```
int x;
```

x é uma variável global.

```
int soma(int a, int b){
```

```
    int result;
```

```
    result = a + b;
```

```
    x = 1000;
```

```
    return result;
```

```
}
```

```
void main(){
```

```
    int a=10, b=5;
```

```
    x = 5;
```

```
    printf("\nO valor de x antes da chamada da funcao e: %d", x);
```

```
    printf("\nO resultado da funcao e: %d", soma(a,b) );
```

```
    printf("\nO valor de x apos a chamada da funcao e: %d", x);
```

```
}
```

Modularidade

Modularidade

Programas de computador podem ser muito grandes e complexos

Dividir um programa em partes menores, torna-o mais fácil de se entender, manter e reutilizar

Essas divisões são denominadas módulos

Em C, as funções de um programa podem ser vistas como módulos

Modularidade

Dessa forma, o ideal é que um programa seja decomposto em funções

Mas, como identificar essas funções?

Aplique a técnica de **refinamentos sucessivos**

A ideia geral é:

- Identifique o macro problema
- Subdivida esse macro problema em subproblemas
- Se os subproblemas ainda forem complexos, subdivida-os também
- Faça isso, até um nível em que a subdivisão seja o mais simples possível de se implementar

Modularidade

Um função deve:

- Ter um nome significativo que facilite identificar o que ela faz
- Realizar um papel bem específico no programa. Ou seja, evite fazer funções que realizam mais de uma coisa

Um exemplo ruim:

```
void leImprimeVetor(int vet[]);
```

Um bom exemplo:

```
void leVetor(int vet[]);
```

```
void imprimeVetor(int vet[]);
```